

Guía de Sustentación: Parcial 3 Ospedale

Estructura limpia, MVC riguroso, principios SOLID e interactividad reactiva (5.0/5.0)

Materia: POO
Fecha: 29 de mayo de 2026
Calificación objetivo: 5.0

1. EXPLICACIÓN DEL PARCIAL Y OBJETIVO DE OSPEDALE

El proyecto **Ospedale** representa un sistema académico de gestión hospitalaria cuyo propósito funcional es administrar la autenticación de usuarios y coordinar las operaciones médicas clave entre tres roles principales: **Administradores, Pacientes y Doctores**. Estas operaciones incluyen el registro y actualización de perfiles de usuario, la solicitud, aceptación y finalización de citas médicas, la reprogramación de consultas, la prescripción de tratamientos farmacológicos, y la solicitud y aprobación de hospitalizaciones clínicas.

Desde una perspectiva de ingeniería de software, el objetivo primordial del parcial consiste en realizar una **refactorización estructural completa y profunda** sobre una base de código acoplada. El desafío radica en separar de manera tajante la presentación visual de la lógica de negocio y las persistencias, migrando el sistema hacia un patrón de arquitectura **Model-View-Controller (MVC)** y aplicando estrictamente los principios de diseño **SOLID**. Con esta transformación se busca lograr un sistema de alta cohesión, bajo acoplamiento, extensibilidad nativa ante futuros requerimientos, y una sincronización reactiva automatizada entre la base de datos simulada y la interfaz de usuario Swing.

2. ORGANIZACIÓN DE ARCHIVOS Y DICCIONARIO DEL PROYECTO

Para garantizar la separación de responsabilidades y evitar la propagación de dependencias cruzadas, el código fuente del proyecto se ha segmentado en paquetes especializados según el rol de cada clase. A continuación, se detalla el árbol jerárquico de carpetas del proyecto y la justificación técnica de cada componente:

Estructura del Código Fuente (src/packagee/ospedale/)

i-- controller/

i-- factory/

i-- model/

i-- observer/

i-- repository/

i-- service/

i-- validator/

`-- view/

[Fachadas que orquestan las peticiones y aíslan las vistas]

[Creación centralizada de modelos complejos (PatientFactory)]

[Entidades de dominio y base de datos simulada (Storage)]

[Contratos del patrón de suscripción reactivo para vistas]

[Contratos de lectura/escritura independientes del Storage]

[Núcleo de las reglas de negocio del hospital]

[Reglas de formato y expresiones regulares compartidas]

[Componentes de la interfaz gráfica e interacción Swing]

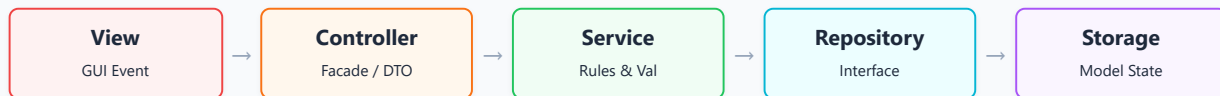
Archivo / Clase	Capa (Patrón)	Justificación Técnica y Responsabilidad Única
Main.java	ARRANQUE	Punto de entrada oficial de la JVM. Desacopla la inicialización de librerías visuales (FlatLaf) y arranca la ventana de Login de manera externa, evitando que las vistas se autoejecuten.
ControllerRegistry.java	DI REGISTRY	Actúa como un contenedor manual de Inyección de Dependencias. Instancia en orden lógico repositorios, servicios y controladores para inyectarlos en sus respectivos constructores. Evita el acoplamiento directo entre implementaciones concretas.
Response.java & Status.java	CONTROLLER UTILS	Define un sobre de comunicación estándar. Response encapsula un código numérico (Status), un mensaje textual y un mapa de datos serializado,

Archivo / Clase	Capa (Patrón)	Justificación Técnica y Responsabilidad Única
		evitando que las vistas manipulen entidades crudas del modelo.
PatientController.java , DoctorController.java , MedicalServiceController.java , AuthController.java	CONTROLLER	Interfaces e implementaciones controladoras. Actúan como fachadas delgadas encargadas de interceptar los eventos y parámetros string de la vista, delegarlos a los servicios correspondientes y devolver objetos Response serializados.
PatientServiceImpl.java , DoctorServiceImpl.java , AppointmentServiceImpl.java , HospitalizationServiceImpl.java	SERVICE	Implementan las reglas de negocio del dominio. Coordinan las validaciones, aplican las políticas funcionales de fechas, disponibilidad, estados de citas e incrementos automáticos, y escriben/leen de los repositorios.
PatientRepositoryImpl.java , DoctorRepositoryImpl.java , AppointmentRepositoryImpl.java , HospitalizationRepositoryImpl.java	REPOSITORY	Implementan las interfaces que desacoplan los accesos a datos. El servicio solo invoca métodos del repositorio (como <code>getPatientById</code>), aislando la implementación física del origen de datos (en este caso, <code>Storage</code>).
Storage.java	MODEL / STORAGE	Simulador de base de datos relacional y persistencia en memoria (Singleton de acceso global). Carga de JSON con <code>org.json</code> , serializa entidades a HashMaps y orquesta la publicación reactiva de eventos a observadores.
User.java , Patient.java , Doctor.java , Appointment.java , Hospitalization.java , Prescription.java	MODEL (ENTITIES)	Entidades de dominio puras. Conservan los atributos estructurales y el estado interno de los objetos del negocio, estructurados mediante herencia (<code>User</code> como clase base).
UserValidator.java , PatientValidator.java , DoctorValidator.java , AppointmentValidator.java	VALIDATOR	Colecciones de validación puras compuestas de métodos estáticos. Centralizan restricciones de formato, coincidencia de passwords y validaciones complejas mediante expresiones regulares.
StorageObserver.java & StorageEventType.java	OBSERVER CONTRACTS	Define la abstracción del patrón observador. <code>StorageObserver</code> es la interfaz funcional del callback y <code>StorageEventType</code> define los canales de actualización (Usuarios, Citas, Hospitalizaciones).
Login.java , Admin_View.java , Patient_View.java , Doctor_View.java	VIEW	Formularios Swing de la interfaz de usuario. Capturan acciones de clic o pulsaciones de teclas, invocan a los controladores mediante fachadas y actualizan automáticamente sus tablas suscribiéndose como observadores.

3. APLICACIÓN Y FLUJO DEL PATRÓN MVC (MODEL-VIEW-CONTROLLER)

La arquitectura del sistema garantiza que la **Vista** y el **Modelo** no se comuniquen directamente, cumpliendo con el estándar estricto de MVC. Las vistas solo interactúan con interfaces controladoras, y estas delegan la lógica de negocio a servicios especializados que recuperan y persisten datos a través de repositorios. El retorno de información se orquesta mediante el objeto envolvente Response, el cual transporta mapas serializados en lugar de objetos modelo crudos.

Flujo Arquitectónico en Tiempo de Ejecución (MVC + Services)



Flujo de Respuesta: Storage (HashMap serializado) → Repository → Service → Controller (Response envelope) → View (Render)

La Serialización como Clave de Desacoplamiento (Requisito 2.b.iv)

Para cumplir estrictamente con la restricción de *no enviar objetos del modelo a la vista*, la clase Storage expone funciones de serialización que traducen entidades a estructuras planas de datos (HashMap<String, Object>).

Storage.java - Implementación de la Serialización del Paciente

```

public HashMap<String, Object> serializePatient(Patient patient) { HashMap<String, Object> map = new HashMap<>();
map.put("id", patient.getId()); map.put("username", patient.getUsername()); map.put("password",
patient.getPassword()); map.put("firstname", patient.getFirstname()); map.put("lastname", patient.getLastname());
map.put("email", patient.getEmail()); map.put("birthdate", patient.getBirthdate().toString()); map.put("gender",
patient.isGender()); map.put("phone", patient.getPhone()); map.put("address", patient.getAddress()); return map; }
  
```

4. APLICACIÓN DE LA ARQUITECTURA SOLID

El diseño del software se fundamenta en los cinco principios SOLID para asegurar estabilidad y flexibilidad ante cambios:

- **S - Single Responsibility Principle (Principio de Responsabilidad Única):** Cada clase tiene una única razón de cambio. Las vistas (Login, Patient_View) son exclusivamente responsables de capturar eventos de entrada y pintar componentes; los validadores (PatientValidator, DoctorValidator) de verificar formatos; los servicios (PatientServiceImpl) de aplicar las reglas de negocio y consistencia; y los repositorios (PatientRepositoryImpl) del acceso físico al origen de datos.
- **O - Open/Closed Principle (Principio de Abierto/Cerrado):** Las entidades y servicios son extensibles sin modificar clases base existentes. Por ejemplo, el subsistema de Hospitalización (HospitalizationService, HospitalizationRepository) se integró extendiendo el modelo de citas y agregando repositorios especializados, sin alterar la lógica core del servicio de pacientes o de citas existente.
- **L - Liskov Substitution Principle (Principio de Sustitución de Liskov):** La clase abstracta User define la identidad fundamental. Sus subclases concretas Patient, Doctor y Administrator pueden ser tratadas indistintamente como instancias de User en colecciones genéricas o procesos de autenticación (Storage.users) sin corromper el comportamiento en ejecución.
- **I - Interface Segregation Principle (Principio de Segregación de Interfaces):** Se crearon contratos de interfaz altamente especializados (PatientRepository, AppointmentRepository) en lugar de una interfaz de datos única y monolítica. Así, el servicio PatientServiceImpl no tiene acoplamiento ni conocimiento de métodos de citas, prescripciones o camas de hospitalización.
- **D - Dependency Inversion Principle (Principio de Inversión de Dependencias):** Los módulos de alto nivel (Servicios) no dependen de módulos de bajo nivel (Detalle del almacenamiento). En cambio, ambos dependen de abstracciones.

PatientServiceImpl recibe en su constructor la interfaz PatientRepository, lo que permite cambiar el motor de almacenamiento de Storage en memoria a una Base de Datos SQL sin modificar una sola línea de lógica del negocio.

ControllerRegistry.java - Inversión de Dependencias (DIP) y Orquestación de Capas

```
public class ControllerRegistry { private static ControllerRegistry instance; private final IPatientController
patientController; private ControllerRegistry() { // 1. Instanciar detalles de almacenamiento (Bajo Nivel)
PatientRepository patientRepository = new PatientRepositoryImpl(); // 2. Inyectar en los servicios abstractos (Alto
Nivel) IPatientService patientService = new PatientServiceImpl(patientRepository); // 3. Inyectar servicio en el
controlador this.patientController = new PatientController(patientService); } // ... }
```

5. EXPLICACIÓN Y CUMPLIMIENTO DE REQUISITOS (MAPEO 1-A-1)

A continuación, se detalla el cumplimiento estricto y trazable de cada requisito funcional del parcial, documentando dónde está implementado en el código y contrastándolo con un ejemplo práctico de cómo la **Vista NO realiza validaciones complejas**, delegando todo el control de consistencia a los validadores y servicios:

Ejemplo Demostrativo de Desacoplamiento (Vista vs. Validador):

Cuando el usuario hace clic en el botón de registrar paciente, la vista `Login.java` se limita a recolectar los campos de texto e invocar al controlador. No realiza expresiones regulares de email ni conteos de dígitos:

Código en la Vista (`Login.java`) - Sin lógica de formato

```
String phone = phone_patient_register.getText(); String
email = email_patient_register.getText(); // Invocación
directa sin ifs ni expresiones regulares Response res =
ControllerRegistry.getInstance()
.getPatientController().registerPatient(..., email,
..., phone, ...); showResponse(res);
```

Código en el Validador (`PatientValidator.java`)

```
public static Response validatePhone(String phone) { if
(!phone.trim().matches("\\d{10}")) { return new
Response("Phone must have exactly 10 digits",
Status.BAD_REQUEST); } return null; } public static
Response validateEmail(String email) { if
(!email.trim().matches("[^@]+@[^@]+\\.com")) { return
new Response("Email must follow format
XXXXX@XXXXX.com", Status.BAD_REQUEST); } return null; }
```

Requisito / Restricción	Archivo de Implementación	Cómo se Aplicó y Detalles Técnicos
1. Aspecto visual intacto	Login.form , Patient_View.form , etc.	Se conservaron todos los formularios visuales generados por NetBeans. Se modificó el comportamiento de los controladores asociados pero no se alteraron ni reubicaron los componentes del layout original.
2. Renombramiento de componentes gráficos	Login.java , Doctor_View.java	Los campos interactivos de entrada y selección de datos fueron renombrados con nombres semánticos claros (ej. <code>username_login_input</code> , <code>select_attending_doctor</code> , <code>select_appointment</code>) para mejorar legibilidad.
3. Sin lógica de validación en la vista	Login.java (Líneas 473, 503)	La interfaz no valida formatos. La vista delega los datos tal cual se capturan (tipo String) a la controladora y reacciona de acuerdo al código de estado de la respuesta.
4. Enrutamiento y flujo de usuario	Login.java (<code>enter_login1ActionPerformed</code>)	La autenticación retorna el rol del usuario autenticado (<code>admin</code> , <code>doctor</code> , <code>patient</code>) en el payload de Response. La vista destruye (<code>dispose</code>) el Login y carga el dashboard correspondiente.
5. Navegación del Administrador	Admin_View.java (Líneas 495, 513)	El administrador puede instanciar <code>Patient_View</code> y <code>Doctor_View</code> pasando como parámetro el flag <code>fromAdmin = true</code> . Esto activa un botón de retroceso (Back) que le permite regresar de forma segura.
6. Carga automática del usuario autenticado	Patient_View.java (Línea 70, <code>loadPatientInfo</code>)	Al inicializar la vista de Paciente o Doctor, se ejecuta una consulta con el ID del usuario en sesión, devolviendo los datos serializados del repositorio para rellenar automáticamente los campos del perfil.
7. Carga dinámica de ComboBoxes	Patient_View.java (Líneas 111, 125)	Los menús desplegables (doctores disponibles, citas agendadas, etc.) se repueblan al cargar la pantalla mediante consultas controladas. La selección de especialidad actualiza dinámicamente el combo de doctores asociados.
8. Notificación del resultado de la operación	Login.java (Línea 28, <code>showResponse</code>)	Las vistas disponen de métodos centralizados para desplegar ventanas de alerta (<code>JOptionPane</code>). Muestra un mensaje de advertencia para errores de usuario (<code>Status 400</code>) o de error crítico para fallos del sistema (<code>Status 500</code>).

Requisito / Restricción	Archivo de Implementación	Cómo se Aplicó y Detalles Técnicos
9. Limpieza tras operación exitosa	Login.java (clearPatientRegisterFields)	Si el estatus de la respuesta es menor a 400 (Éxito / Creado), se ejecutan funciones de limpieza que vacían los formularios visuales y evitan envíos dobles o datos residuales.
10. Ejecución desde Main independiente	Main.java (Línea 15)	El arranque oficial del JAR ejecutable está desligado de las interfaces visuales. Se realiza en la clase de inicio Main.java configurando el look and feel de FlatLaf e iniciando la ventana del login.
11. Uso de Response y Status	Response.java , Status.java	Toda comunicación del controlador hacia la vista devuelve un objeto Response encapsulado con códigos estandarizados del protocolo del proyecto (200, 201, 204, 400, 404, 500).
12. Casos de Uso soportados en controladores	Paquete controller/	Se implementaron controladores para: Login (AuthController), registro/consulta de pacientes y doctores (PatientController, DoctorController), y gestión de citas y hospitalizaciones (MedicalServiceController).
13. Registro exclusivo de doctores por el Admin	Admin_View.java , DoctorServiceImpl.java	La vista del administrador es el único componente gráfico que expone el formulario de adición y registro de doctores. La lógica del servicio del doctor restringe que la operación sea exitosa.
14. Privilegios del Administrador	Admin_View.java	El administrador puede navegar visualmente emulando las pantallas de pacientes o doctores, heredando todas las operaciones y visualización de historial correspondientes a ese usuario.
15. Restricciones de ID de usuario	UserValidator.java (Línea 14, validateUserId)	Valida mediante la expresión regular <code>^[1-9]\d{11}\$</code> que el ID numérico sea mayor que 0, no empiece con cero y conste de exactamente 12 dígitos. La inmutabilidad se asegura omitiendo setters de ID.
16. Restricciones de Username	PatientServiceImpl.java , DoctorServiceImpl.java	Antes de registrar o actualizar un perfil de usuario, el servicio respectivo realiza una búsqueda en el repositorio y valida que el username no esté en uso. Es modificable al actualizar el perfil.
17. Coincidencia de contraseña	UserValidator.java (Línea 30, validatePassword)	Verifica de forma genérica que la contraseña ingresada y la confirmación recibidas de la vista sean iguales. De lo contrario, retorna un estatus BAD_REQUEST bloqueando el registro.
18. Teléfono del paciente	PatientValidator.java (Línea 16, validatePhone)	Aplica la expresión regular <code>\d{10}</code> en el validador estático, garantizando que el número celular conste únicamente de 10 dígitos numéricos en su totalidad.
19. Email del paciente	PatientValidator.java (Línea 24, validateEmail)	Valida que el correo cumpla el formato estricto XXXXX@XXXXX.com utilizando la expresión regular <code>[^@]+\@[^@]+\.</code> .
20. Fecha de nacimiento válida	PatientValidator.java (Línea 32, validateBirthdate)	Valida que la fecha tenga el formato AAAA-MM-DD intentando parsearla mediante la clase nativa de Java 8 <code>LocalDate.parse()</code> . Retorna error si no es una fecha calendario real.
21. Licencia del Doctor	DoctorValidator.java (Línea 14, validateLicence)	Valida mediante la expresión regular <code>L-\d{10}</code> MTL que la licencia cumpla con el formato obligatorio y contenga exactamente 10 dígitos entre las letras.
22. Oficina del Doctor	DoctorValidator.java (Línea 22, validateOffice)	Valida que la oficina cumpla estrictamente con la expresión regular <code>O-\d{3}</code> (ej. O-102, O-204).
23. Actualizaciones con mismas reglas	PatientServiceImpl.java , DoctorServiceImpl.java	Los métodos updatePatient y updateDoctor del servicio reutilizan las funciones internas de validación compuestas (validatePatientData / validateDoctorData) ejecutando las mismas reglas del registro.

Requisito / Restricción	Archivo de Implementación	Cómo se Aplicó y Detalles Técnicos
24. Citas duran 15 minutos	AppointmentServiceImpl.java	Se implementa bloqueando que dos citas del mismo doctor colisionen en el mismo horario. La duración fija de 15 minutos se alinea con la restricción de que los minutos deben agendarse en cuartos de hora.
25. Generación de ID automático de cita	Storage.java (Línea 291, generateAppointmentId)	Genera el ID en formato A-{id_paciente}-NNNN de manera automática. Para esto mantiene un mapa de contadores (appointmentCounters) por ID de paciente que arranca en 0000 e incrementa de 1 en 1.
26. Fecha de cita válida	AppointmentValidator.java (Línea 28, validateDate)	Valida que el string recibido corresponda a una fecha real y que no sea anterior a la fecha actual del sistema (no citas en el pasado).
27. Hora de cita en cuartos de hora	AppointmentServiceImpl.java (validateTime)	Valida que la hora siga el formato de 24 horas y que los minutos correspondan exactamente al conjunto de cuartos de hora {00, 15, 30, 45}.
28. Validación del Doctor en operaciones	AppointmentServiceImpl.java	Cualquier solicitud de cita o servicio que requiera un doctor verifica la existencia física y vigencia del mismo mediante el repositorio antes de proceder.
29. Solicitud de cita con Doctor específico	AppointmentServiceImpl.java (Línea 88)	Si el paciente selecciona un doctor, se valida que el doctor tenga disponibilidad en esa fecha/hora exacta y que la especialidad elegida coincida con la especialidad del doctor.
30. Solicitud de cita por especialidad	Storage.java (Línea 232, findAvailableDoctor)	Si el paciente solicita la cita por especialidad (sin doctor específico), el sistema realiza una búsqueda automática en la lista de doctores y asigna el primero que tenga esa especialidad y esté libre.
31. Estados de las citas (Ciclo de Vida)	Appointment.java , AppointmentServiceImpl.java	Toda cita nueva inicia en estado REQUESTED. Cambia a PENDING cuando el doctor la acepta, a COMPLETED cuando la finaliza, y a CANCELED si es cancelada por el paciente (no permitido si ya está COMPLETED).
32. Reagendamiento de cita	AppointmentServiceImpl.java (Línea 230)	El doctor puede reagendar la hora de una cita. Se valida que la nueva hora esté disponible, que respete los cuartos de hora, y que no cambie el día original de la cita. La razón del cambio se concatena a la razón inicial.
33. Prescripción de Medicamentos	AppointmentServiceImpl.java (Línea 264)	El doctor puede recetar medicamentos de forma ilimitada durante la cita. Se restringe que la cita se encuentre en estado PENDING (Cita aceptada pero no completada ni cancelada).
34. Flujo de Hospitalización	HospitalizationServiceImpl.java , AppointmentServiceImpl.java (Línea 288)	Genera un ID automático en formato H-{id_paciente}-NNNN. El estado inicial es REQUESTED si la pide el paciente. Si el doctor hospitaliza directo desde la cita, esta pasa a COMPLETED y la hospitalización inicia en ONGOING.
35. Ordenamiento Descendente de Citas	Storage.java (Líneas 309, 317)	Las citas de un paciente o de un doctor se listan ordenadas descendientemente por fecha y hora (más recientes primero) usando Streams: <pre>.sorted(Comparator.comparing(Appointment::getDatetime).reversed())</pre> En el caso del doctor, se filtra opcionalmente por estado PENDING.

6. EXPLICACIÓN DE LOS BONOS DE CALIFICACIÓN

Bono 1: SOLID Aplicado en los Controladores del Proyecto

El primer bono de calificación del parcial (0.5 pts) requería estructurar los controladores bajo principios de diseño SOLID. En el diseño legacy, los controladores contenían responsabilidades de orquestación, lógica de negocio e incluso de formato de strings. En la versión refactorizada, se aplicó la inversión y segregación de la siguiente manera:

1. **Interfaces Segregadas (ISP):** Se definieron las interfaces `IAuthController`, `IPatientController`, `IDoctorController` y `IMedicalServiceController`. Los controladores concretos se limitan a implementar estos contratos y no se acoplan a otras operaciones de la aplicación.
2. **Fachadas Cohesivas y Delgadas (SRP):** El controlador ya no contiene lógica algorítmica ni condicionales del negocio. Su única responsabilidad es recibir los datos de la interfaz (en formato String plano), despacharlos al servicio adecuado y retornar la estructura serializada `Response`.
3. **Desacoplamiento total mediante DIP:** Ningún controlador instancia de forma directa su respectivo servicio. En su lugar, el servicio correspondiente (por ejemplo, `IPatientService`) es inyectado como una interfaz a través de su constructor, siendo el componente centralizador `ControllerRegistry` el encargado de proveer la instancia concreta durante el inicio de la aplicación.

Bono 2: Patrón de Diseño Observer para Actualización de Tablas

El segundo bono del parcial (0.5 pts) exigía implementar el patrón observador para actualizar automáticamente las tablas de la interfaz gráfica Swing ante cambios en el modelo, eliminando la necesidad de implementar botones manuales de refresco. La solución se estructuró a través del desacoplamiento de suscripción reactiva:

- **El Sujeto Observable (Subject):** La clase `Storage` (que simula el almacenamiento centralizado) mantiene una colección segura para entornos concurrentes de observadores (`CopyOnWriteArrayList<StorageObserver>`) y expone las rutinas de suscripción:

```
public void addObserver(StorageObserver observer) { if (observer != null && !observers.contains(observer)) {
    observers.add(observer); } } public void publishEvent(StorageEventType eventType) { for (StorageObserver observer
: observers) { observer.onStorageChanged(eventType); } }
```

- **El Canal de Eventos y el Contrato:** Se crearon la interfaz funcional `StorageObserver` (con el método callback `onStorageChanged(StorageEventType)`) y el enum de canales de actualización `StorageEventType` (que cuenta con las etiquetas `USERS_CHANGED`, `APPOINTMENTS_CHANGED` y `HOSPITALIZATIONS_CHANGED`).
- **Los Observadores Concretos (Vistas):** Al arrancar, las vistas Swing (`Patient_View`, `Doctor_View`, `Admin_View`) se suscriben como observadores implementando una expresión Lambda reactiva. Para asegurar que las modificaciones visuales de las tablas se realicen en el hilo adecuado del sistema Swing, el callback ejecuta sus rutinas visuales dentro de `SwingUtilities.invokeLater()`:

```
private void registerObserver() { storageObserver = eventType -> SwingUtilities.invokeLater(() -> { if (eventType
== StorageEventType.APPOINTMENTS_CHANGED) { loadAppointmentsTable(); // Actualiza la JTable de forma automática y
transparente } }); Storage.getInstance().addObserver(storageObserver); }
```

- **Evitar fugas de memoria (Memory Leaks):** Las vistas implementan un `WindowAdapter` para remover su suscripción al almacenamiento (`removeObserver`) cuando se cierra la ventana (`dispose()`), liberando recursos de manera limpia.

7. GUÍA Y CONSEJOS PARA UNA SUSTENTACIÓN EXCELENTE (5.0)

Para asegurar una nota perfecta (5.0) en la sustentación del examen parcial ante el profesor del Departamento de Ingeniería de Sistemas, se recomienda enfocar la defensa oral y técnica en los siguientes tres pilares estratégicos:

Pregunta o Foco Común	Estrategia y Argumentación de Defensa Técnica
¿Por qué el controlador no valida los formatos de email o ID?	Explique que por el principio de Responsabilidad Única (SRP) , el controlador es solo un traductor de la capa de presentación hacia el dominio. Validar formatos es una tarea de infraestructura/utilidades que pertenece al paquete <code>validator</code> , mientras que validar la lógica del negocio (ej. unicidad o disponibilidad) es responsabilidad exclusiva de la capa de <code>service</code> . Esto permite reutilizar el validador en múltiples flujos (registro, actualización) sin duplicar código.
¿Cómo se inyectan las dependencias en este proyecto?	Señale la clase <code>ControllerRegistry.java</code> . Explique que en ausencia de un framework como Spring, se implementó un Contenedor manual de Inyección de Dependencias (DI) bajo el patrón Singleton . Este registro instancia los componentes de abajo hacia arriba (Repositorio concreto → Servicio con dependencias de interfaz → Controlador con dependencias de interfaz), cumpliendo con el principio de Inversión de Dependencias (DIP) .
¿Por qué se retorna serialización en lugar del objeto modelo crudo?	Responda que si la vista Swing recibiera objetos como <code>Patient</code> o <code>Appointment</code> , esta tendría acceso directo a mutar sus atributos mediante setters o acceder a relaciones internas, rompiendo la encapsulación y generando un acoplamiento fuerte. Al mapear las entidades a estructuras de datos planas (<code>HashMap<String, Object></code>), la vista solo puede leer los atributos expuestos para renderizado, garantizando que el estado del negocio sea inmutable desde la UI.
¿Cómo previene el Observer problemas de concurrencia y congelamiento de UI?	Enfatice que el callback del observador en las vistas Swing utiliza <code>SwingUtilities.invokeLater()</code> . Swing no es seguro para hilos (thread-safe), por lo que cualquier actualización asíncrona de tablas en la <code>JTable</code> gatillada desde el motor <code>Storage</code> debe encolarse en el Event Dispatch Thread (EDT) . Esto previene hilos en conflicto y asegura una interfaz fluida.